



Rede “Raízes do Nordeste” — Sistema de Pedidos Multicanal

Projeto Multidisciplinar — Trilha Back-end

Ramon De Oliveira

Sumário

1. Introdução	2
2. Análise e Requisitos	2
3. Modelagem e Arquitetura	7
4. Entrega Técnica	13
5. Plano de Testes e Evidências	13
6. Conclusão	15
7. Referências	16
Anexos — Evidências de Execução	16

Curso: CST em Análise e Desenvolvimento de Sistemas

Disciplina: Projeto Multidisciplinar

Aluno: Ramon De Oliveira

RU: 4904087

1. Introdução

Contexto

A Raízes do Nordeste é uma rede de lanchonetes de culinária nordestina que nasceu como um negócio familiar em Recife e hoje está em expansão por capitais e cidades do interior do Brasil. Com o crescimento, a empresa decidiu reestruturar seus sistemas para sustentar uma operação com múltiplas unidades, diversos canais de atendimento e exigências de padronização, rastreabilidade e conformidade com a LGPD.

Objetivos do projeto

- Oferecer uma jornada única de pedido em todos os canais: aplicativo oficial, totens de autoatendimento, balcão e retirada rápida (pick-up);
- Controlar cardápio, estoque e regras específicas de cada unidade (incluindo produtos sazonais, como os do período junino, e variações regionais de receitas);
- Integrar pagamentos com serviços externos, sem processar dados de cartão;
- Implantar programa de fidelidade com pontos e descontos progressivos, respeitando a LGPD;
- Fornecer à matriz relatórios de vendas, indicadores e auditoria de operações sensíveis.

Principais usuários

- **Cliente** — consulta o cardápio, faz pedidos e acompanha o status;
- **Atendente** — registra pedidos no balcão;
- **Gerente da unidade** — gerencia cardápio, estoque e operações da loja;
- **Matriz (franqueadora)** — acompanha relatórios, indicadores e auditoria;
- **Serviço externo de pagamento** — processa os pagamentos e devolve o resultado.

Relevância do sistema

Nos horários de pico, indisponibilidade impacta diretamente faturamento, operação e imagem da marca. O sistema é, portanto, peça central do negócio: precisa ser robusto, escalável, auditável e tolerante a falhas, além de garantir o uso responsável dos dados dos clientes.

2. Análise e Requisitos

Documento produzido a partir da leitura integral do estudo de caso oficial da disciplina Projeto Multidisciplinar (Trilha Back-end). Todas as afirmações abaixo referenciam a seção do estudo de caso que as fundamenta.

Etapas 1 — Leitura

O estudo de caso foi lido integralmente. Resumo do contexto (seções 1 a 7):

A Raízes do Nordeste é uma rede de lanchonetes de culinária nordestina, nascida em Recife e em expansão para várias capitais e cidades do interior (seção 1). O cliente interage por múltiplos canais — aplicativo oficial, totens de autoatendimento, balcão e retirada rápida (pick-up) — e espera a mesma jornada em todos eles: ver o cardápio da unidade, selecionar produtos disponíveis naquele local, pedir, acompanhar o status e receber corretamente (seção 2). As unidades diferem entre si (cozinha completa ou reduzida, produtos sazonais — especialmente juninos — e variações regionais de receita), diferenças que devem ser controladas sem quebrar o padrão da franquia (seção 2). Cada unidade tem equipe própria, estoque local, regras de funcionamento e metas (seção 3). A matriz acompanha vendas por unidade/região, produtos mais consumidos, relatórios financeiros, dados consolidados e auditoria de operações sensíveis — cancelamentos, descontos e ajustes (seção 3). Há um programa de fidelização (pontos, descontos progressivos, campanhas segmentadas) com conformidade estrita à LGPD: consentimento explícito, tratamento adequado, anonimização, auditoria de acessos (seção 4). Pagamentos são processados por serviços externos: o sistema apenas solicita, recebe

confirmação/negativa, registra e atualiza o pedido (seção 5). O sistema deve ser robusto, escalável, auditável, tolerante a falhas e de alta disponibilidade (seção 6), com arquitetura coerente e sustentável (seção 7).

Etapa 2 — Requisitos explícitos

Requisitos Funcionais (RF)

ID	Requisito	Fonte no estudo de caso
RF01	Exibir o cardápio da unidade selecionada, contendo apenas produtos disponíveis naquele local.	Seção 2 (“visualizar o cardápio da unidade”; “selecionar produtos disponíveis naquele local”)
RF02	Registrar pedidos originados de qualquer canal: aplicativo, totem, balcão (atendimento humano) e pick-up.	Seção 2 (lista de canais)
RF03	Permitir pedidos antecipados pelo aplicativo.	Seção 2 (“Aplicativo oficial (pedidos antecipados...)”)
RF04	Permitir acompanhamento do status do pedido até a entrega/retirada.	Seção 2 (“acompanhar o status”; “receber o pedido corretamente”)
RF05	Controlar a disponibilidade de produtos por unidade, incluindo produtos sazonais (ex.: período junino) e variações regionais, sem quebrar o padrão da franquia.	Seção 2 (parágrafo final)
RF06	Controlar estoque local por unidade.	Seção 3 (“controle de estoque local”)
RF07	Registrar auditoria de operações sensíveis: cancelamentos, descontos e ajustes.	Seção 3 (“auditoria de operações sensíveis (cancelamentos, descontos, ajustes)”)
RF08	Disponibilizar à matriz relatórios de vendas por unidade e por região.	Seção 3 (“vendas por unidade e por região”)
RF09	Disponibilizar relatório de produtos mais consumidos.	Seção 3 (“produtos mais consumidos”)
RF10	Disponibilizar relatórios financeiros e dados consolidados para decisões estratégicas.	Seção 3
RF11	Programa de fidelidade: acumular pontos por consumo.	Seção 4 (“acumular pontos”)
RF12	Programa de fidelidade: oferecer descontos progressivos.	Seção 4 (“descontos progressivos”)

ID	Requisito	Fonte no estudo de caso
RF13	Programa de fidelidade: permitir campanhas segmentadas considerando frequência de consumo, idade e perfil do cliente.	Seção 4
RF14	Registrar consentimento explícito do cliente para uso de seus dados (opt-in).	Seção 4 (“consentimento explícito para uso de dados”)
RF15	Permitir anonimização de dados pessoais quando aplicável.	Seção 4 (“anonimização quando aplicável”)
RF16	Registrar auditoria de acessos a dados pessoais.	Seção 4 (“auditoria de acessos”)
RF17	Solicitar pagamento a serviço externo, receber confirmação ou negativa, registrar o resultado e atualizar o status do pedido.	Seção 5 (lista completa)
RF18	Tratar falhas na integração de pagamento sem corromper o estado do pedido.	Seção 5 (“exige cuidado na modelagem e no tratamento de falhas”)
RF19	Cadastrar e gerenciar unidades da franquia com suas características (formato de cozinha, regras de funcionamento).	Seção 2/3 (“Nem todas as unidades são idênticas”; “regras de funcionamento específicas”)
RF20	Acompanhar metas e indicadores de desempenho por unidade.	Seção 3 (“metas e indicadores de desempenho”)

Requisitos Não Funcionais (RNF)

ID	Requisito	Fonte
RNF01	Alta disponibilidade, principalmente em horários de pico.	Seção 6
RNF02	Escalabilidade para suportar o crescimento contínuo da rede.	Seções 1, 3, 6
RNF03	Tolerância a falhas (inclusive falhas do serviço externo de pagamento).	Seções 5, 6
RNF04	Auditabilidade: rastreabilidade e transparência das operações.	Seções 3, 6
RNF05	Conformidade com a LGPD (consentimento, minimização, anonimização, auditoria de acesso).	Seção 4
RNF06	Segurança: pagamentos desacoplados; o sistema não processa nem armazena dados de cartão.	Seção 5
RNF07	Padronização entre unidades preservando variações controladas (sazonais/regionais).	Seção 2
RNF08	Arquitetura coerente, sustentável no longo prazo e baseada em integração de sistemas.	Seções 5, 7

ID	Requisito	Fonte
RNF09	Consistência multicanal: a mesma jornada em app, totem, balcão e pick-up.	Seção 2

Etapa 3 — Inferências mínimas (justificadas)

Somente inferências indispensáveis para o sistema funcionar, cada uma ancorada no texto:

ID	Inferência	Justificativa
INF01	O ciclo de vida do pedido possui estados: CRIADO → AGUARDANDO_PAGAMENTO → PAGO → EM_PREPARO → PRONTO → ENTREGUE, com CANCELADO e PAGAMENTO_RECUSADO como desvios.	O estudo exige “acompanhar o status” (seção 2) e atualização de status conforme resultado do pagamento (seção 5); cancelamento é citado como operação sensível auditável (seção 3). Estados mínimos para cobrir essas exigências.
INF02	Existe cadastro de cliente vinculado ao programa de fidelidade, com dados mínimos (nome, e-mail, data de nascimento) e flag de consentimento.	A fidelização considera “frequência de consumo, idade e perfil” (seção 4) — exige identificar o cliente e sua data de nascimento; a LGPD exige consentimento registrado. Pedidos de balcão/totem podem ser anônimos (nada no estudo obriga identificação).
INF03	A regra de desconto progressivo é parametrizada por faixas de pontos acumulados.	“Descontos progressivos” (seção 4) exige alguma progressão; faixas de pontos é a materialização mínima, sem inventar regras adicionais. Valores das faixas são configuráveis.
INF04	A baixa de estoque ocorre quando o pedido é confirmado (pago), por unidade.	“Controle de estoque local” (seção 3) só faz sentido se o consumo dos pedidos abater o estoque da unidade.
INF05	O sistema solicita o pagamento ao serviço externo e recebe o resultado depois, por um endpoint de retorno (na implementação, o serviço externo é simulado).	Seção 5 descreve exatamente esse fluxo (solicita → recebe confirmação/negativa → registra → atualiza), e afirma que o processamento é externo. O serviço real não faz parte do escopo.
INF06	Perfis de acesso mínimos: CLIENTE, ATENDENTE, GERENTE (unidade) e MATRIZ.	Seção 3 cita equipes (atendentes, cozinheiros, gerentes) e necessidades exclusivas da matriz (relatórios consolidados, auditoria); relatórios e auditoria não podem ser públicos (rastreadabilidade/transparência, LGPD).
INF07	A anonimização apaga os dados pessoais do cliente de forma definitiva, mas mantém os registros de pedidos para os relatórios.	Seção 4 pede anonimização “quando aplicável” e a matriz continua precisando de dados consolidados (seção 3).
INF08	Os registros de auditoria guardam autor, operação, data e justificativa, e não podem ser alterados nem apagados.	“Rastreabilidade e transparência” (seção 3) e “auditoria de acessos” (seção 4) exigem esses atributos mínimos.

Etapa 4 — Validação

Checklist de aderência: cada funcionalidade implementada rastreia para um RF/RNF acima, e cada RF/RNF rastreia para uma seção do estudo de caso. Funcionalidades deliberadamente **fora do escopo** por não terem respaldo no estudo de caso:

- Delivery/entrega em domicílio (o estudo cita apenas pick-up e consumo em loja);
- Processamento/armazenamento de dados de cartão (explicitamente delegado a terceiros);
- Gestão de folha de pagamento/RH das equipes;
- Cadastro de fornecedores e compras (o estudo cita apenas o controle do estoque local);
- App mobile e interface de totem em si (trilha Back-end: o sistema expõe a API que atende todos os canais).

Nenhuma funcionalidade além das listadas em RF/RNF/INF foi incluída na modelagem ou na implementação.

3. Modelagem e Arquitetura

Diagrama de Casos de Uso

Diagrama de Casos de Uso — Rede Raízes do Nordeste

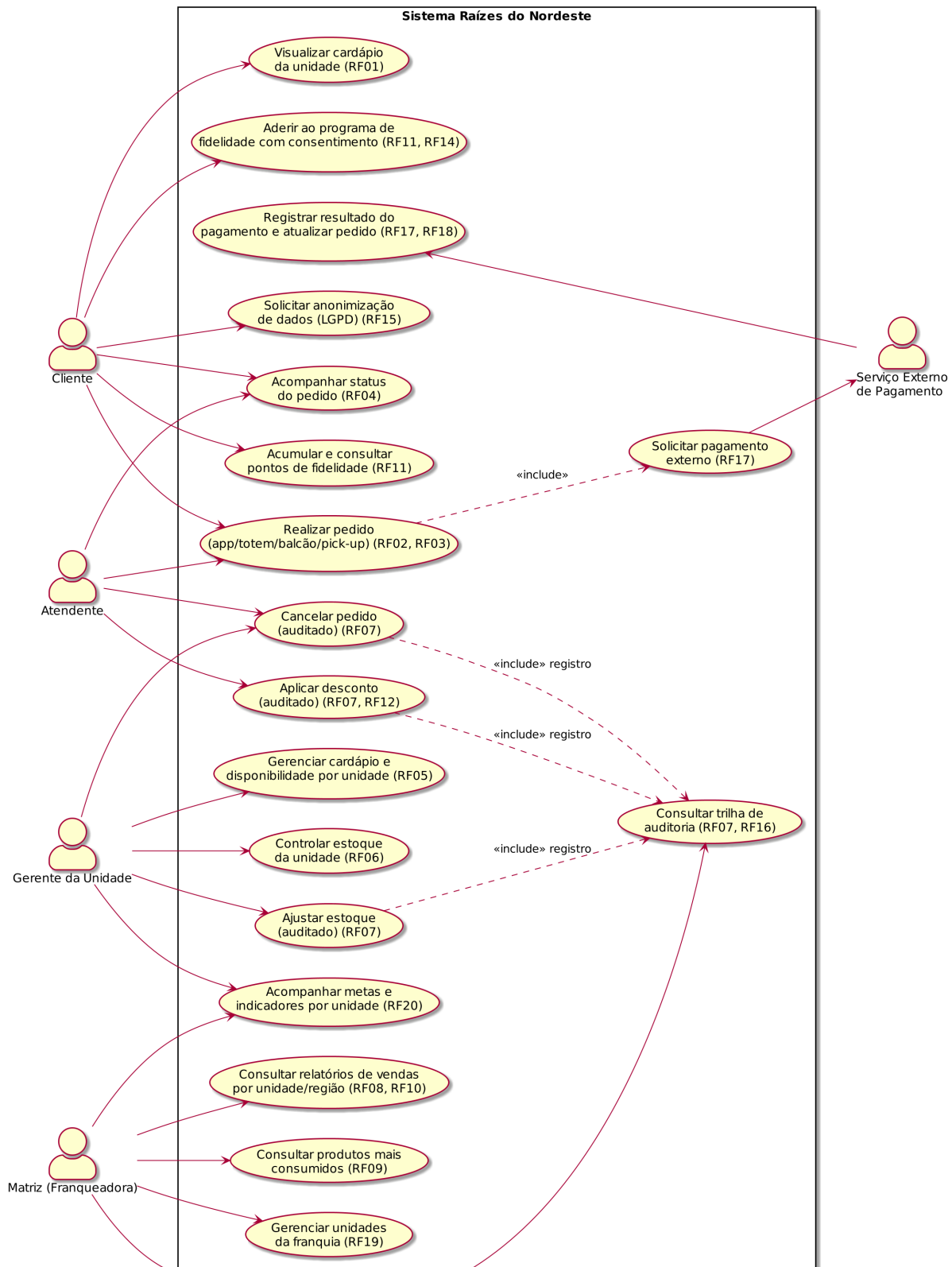


Figura 1: Diagrama de Casos de Uso

Diagrama de Classes

Diagrama de Classes (domínio) — Rede Raízes do Nordeste

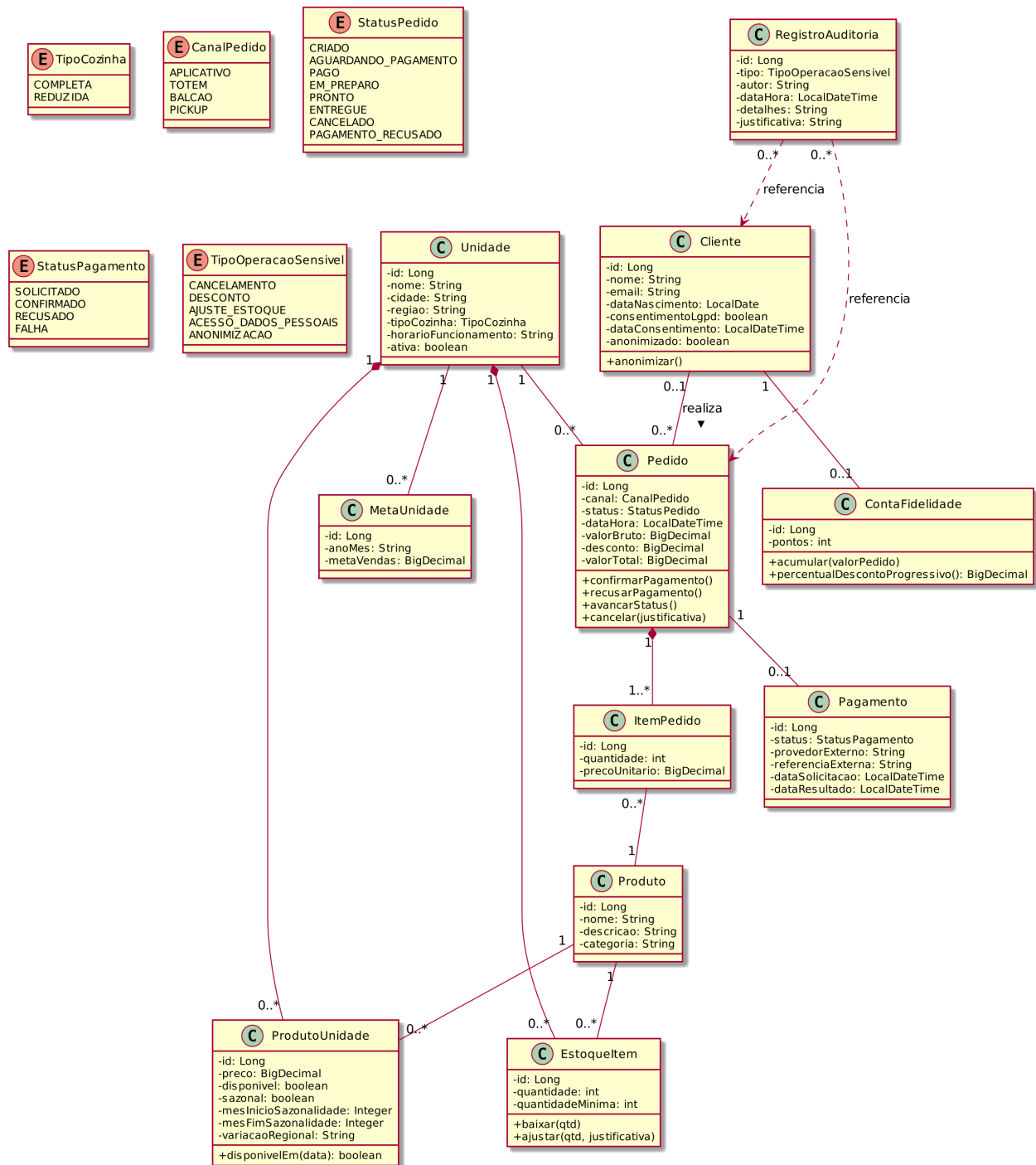


Figura 2: Diagrama de Classes

DER

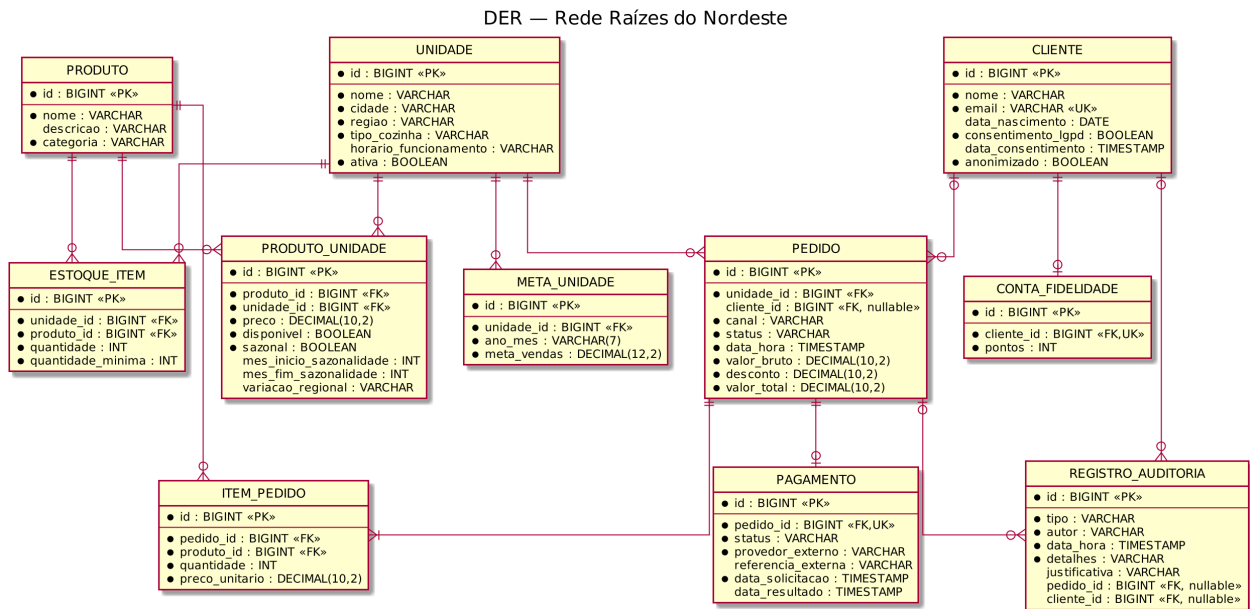


Figura 3: DER

Arquitetura Proposta

Visão geral

A solução é um **back-end único (API REST)** que atende todos os canais descritos no estudo de caso — aplicativo oficial, totens de autoatendimento, atendimento de balcão e pick-up (seção 2). Assim, todos os canais usam as mesmas regras de negócio e os mesmos dados, e o cliente tem a mesma experiência em qualquer um deles.

Os quatro canais (aplicativo, totem, balcão e pick-up) fazem requisições à mesma API, que é organizada em três camadas (Controller, Service e Repository) e usa um banco de dados relacional. A API também se comunica com o serviço externo de pagamento (seção 5) e grava os registros de auditoria no banco.

Decisões e justificativas

Decisão	Justificativa (estudo de caso)
API REST organizada em camadas (Controller, Service, Repository)	Separa as regras de negócio do restante do código, deixando o sistema mais organizado e fácil de manter (seção 7).
Uma única aplicação, dividida em módulos (cardápio, pedidos, estoque, fidelidade, pagamentos, auditoria, relatórios)	É a opção mais simples de desenvolver e manter para o porte atual da rede; no futuro, com o crescimento, os módulos podem virar serviços separados (seções 1 e 6).

Decisão	Justificativa (estudo de caso)
Pagamento feito por serviço externo, através de uma interface (PagamentoGateway)	Seção 5: o sistema apenas solicita o pagamento e recebe o resultado. Usando uma interface, é possível trocar o provedor sem mexer no resto do código; na entrega acadêmica o provedor é simulado.
O resultado do pagamento chega por um endpoint de retorno (callback) protegido contra duplicação	Seção 5 pede cuidado no tratamento de falhas: se o serviço externo enviar o mesmo resultado duas vezes, o sistema percebe e não baixa o estoque nem soma pontos em dobro.
Registros de auditoria que só podem ser adicionados (nunca alterados ou apagados)	Seções 3 e 4: cancelamentos, descontos, ajustes e acessos a dados pessoais precisam ficar registrados com autor, data e justificativa.
Banco de dados relacional (H2 na entrega; PostgreSQL sugerido para produção)	Os dados são bem relacionados entre si (pedidos, itens, estoque por unidade) e operações como baixa de estoque precisam de transações (seções 3 e 5).
Dados de cartão nunca passam pelo sistema	Seção 5: o processamento é 100% externo — menos risco de segurança.
Perfis de acesso (cliente, atendente, gerente, matriz)	Seção 3 (equipes e matriz) + LGPD (seção 4): relatórios e auditoria não podem ser públicos.

Alta disponibilidade, escalabilidade e tolerância a falhas (seção 6)

- A API não guarda estado na memória entre requisições, então é possível rodar várias cópias dela ao mesmo tempo atrás de um balanceador de carga — nos horários de pico, basta adicionar mais cópias;
- Os relatórios da matriz podem ler de uma cópia do banco, para não atrapalhar a operação das lojas nos horários de pico;
- Se o serviço de pagamento falhar, o pedido fica como AGUARDANDO_PAGAMENTO e a solicitação pode ser feita de novo, sem corromper nenhum dado;
- As chamadas ao serviço externo têm limite de tempo (timeout) e podem ser repetidas em caso de erro;
- Os logs da aplicação e os registros de auditoria permitem rastrear o que aconteceu no sistema (seção 3).

LGPD (seção 4)

- O consentimento do cliente é registrado com data e hora antes de qualquer uso dos dados para fidelidade ou campanhas;
- O cliente pode pedir a anonimização: seus dados pessoais são apagados de forma definitiva, mas os pedidos continuam existindo para os relatórios da matriz (sem identificar ninguém);
- Todo acesso a dados pessoais e toda anonimização geram um registro de auditoria;
- São guardados apenas os dados necessários: nome, e-mail e data de nascimento (usados pela fidelidade e pelas campanhas por idade/perfil/frequência — seção 4).

Evolução futura

O crescimento da rede (seções 1 e 6) é suportado sem grandes mudanças: novos canais podem consumir a mesma API e, se necessário, os módulos podem ser separados em serviços independentes no futuro.

Documentação da API (principais endpoints)

A documentação interativa (OpenAPI/Swagger) é gerada automaticamente pelo back-end:

- Swagger UI: <http://localhost:8080/swagger-ui.html>
- OpenAPI JSON: <http://localhost:8080/v3/api-docs>

Todos os canais do estudo de caso (aplicativo, totem, balcão, pick-up) consomem esta mesma API, indicando o canal no campo `canal` do pedido.

Unidades e cardápio (RF01, RF05, RF19)

Método	Rota	Descrição
GET	<code>/api/unidades</code>	Lista as unidades da rede
GET	<code>/api/unidades/{id}</code>	Detalha uma unidade
POST	<code>/api/unidades</code>	Cadastra unidade (matriz)
GET	<code>/api/unidades/{id}/cardapio</code>	Cardápio da unidade — só produtos disponíveis no local e no período (sazonalidade junina, variações regionais)

Pedidos (RF02, RF03, RF04, RF07, RF12)

Método	Rota	Descrição
POST	<code>/api/pedidos</code>	Cria pedido de qualquer canal; valida cardápio/estoque da unidade, aplica desconto progressivo de fidelidade e solicita o pagamento externo
GET	<code>/api/pedidos/{id}</code>	Acompanha o pedido e seu status
GET	<code>/api/pedidos?unidadeId={id}</code>	Lista pedidos da unidade
POST	<code>/api/pedidos/{id}/avancar</code>	Avança status: PAGO → EM_PREPARO → PRONTO → ENTREGUE
POST	<code>/api/pedidos/{id}/cancelar</code>	Cancela operação sensível — auditada; devolve estoque se já pago
POST	<code>/api/pedidos/{id}/desconto</code>	Desconto manual (operação sensível — auditada)

Exemplo — criar pedido:

```
POST /api/pedidos
{
  "unidadeId": 1,
  "clienteId": 1,
  "canal": "APLICATIVO",
  "itens": [ { "produtoId": 1, "quantidade": 2 } ]
}
```

Ciclo de vida do pedido (INF01): CRIADO → AGUARDANDO_PAGAMENTO → PAGO → EM_PREPARO → PRONTO → ENTREGUE, com desvios CANCELADO e PAGAMENTO_RECUSADO.

Pagamentos (RF17, RF18)

Método	Rota	Descrição
POST	/api/pagamentos/{pedidoId}/resultado	Resultado externo com CONFIRMADO ou RECUSADO. Se o mesmo resultado for enviado duas vezes, o sistema ignora a repetição

POST /api/pagamentos/1/resultado

```
{ "referenciaExterna": "EXT-abc", "status": "CONFIRMADO" }
```

O sistema nunca recebe dados de cartão — apenas o resultado do processamento externo (estudo de caso, seção 5).

Cientes, fidelidade e LGPD (RF11–RF16)

Método	Rota	Descrição
POST	/api/clientes	Cadastro (com consentimento explícito opcional)
GET	/api/clientes/{id}	Consulta dados pessoais — exige header X-Autor e gera auditoria de acesso
POST	/api/clientes/{id}/consentimento	Envia consentimento explícito e abre a conta de fidelidade
POST	/api/clientes/{id}/anonimizacao	Anonimização definitiva dos dados do cliente (auditada) — exige header X-Autor
GET	/api/clientes/{id}/fidelidade	Pontos, percentual de desconto progressivo e frequência de consumo

Regras de fidelidade (INF03): 1 ponto por R\$1 em pedidos confirmados; desconto progressivo por faixa — 100+ pontos: 5%; 500+: 10%; 1000+: 15%. Pontos e desconto só se aplicam a clientes **com consentimento LGPD**.

Estoque (RF06, RF07)

Método	Rota	Descrição
GET	/api/estoque?unidadeId={unidadeId}	Estoque local da unidade
POST	/api/estoque/{itemId}/ajuste	Ajuste manual (operação sensível — auditada)

Relatórios da matriz (RF08, RF09, RF10, RF20)

Método	Rota	Descrição
GET	/api/relatorios/vendas?unidadeId={unidadeId}	Vendas consolidadas por unidade
GET	/api/relatorios/vendas?regiao={regiao}	Vendas consolidadas por região
GET	/api/relatorios/producao?produtoId={produtoId}	Produção consolidada
GET	/api/relatorios/indicadores?unidadeId={unidadeId}	Métricas consolidadas por unidade

Auditoria (RF07, RF16)

Método	Rota	Descrição
GET	/api/auditoria?tipo=	Trilha de auditoria; filtro por tipo (CANCELAMENTO, DESCONTO, AJUSTE_ESTOQUE, ACESSO_DADOS_PESSOAIS, ANONIMIZACAO)

Erros

HTTP	Situação
400	Validação de entrada (Bean Validation)
404	Recurso não encontrado
422	Regra de negócio violada (produto fora do cardápio da unidade, estoque insuficiente, status inválido...)

4. Entrega Técnica

Repositório

Todo o código-fonte, a documentação e os diagramas estão versionados em:

<https://github.com/Ramo-n/raizes-do-nordeste-backend>

Artefatos entregues

- Código do back-end (Java 17 + Spring Boot 3) — pasta `backend/src/main/java`
- Testes automatizados (12 testes, JUnit 5) — pasta `backend/src/test/java`
- Massa de dados de exemplo — arquivo `backend/src/main/resources/data.sql`
- Levantamento de requisitos — `documentacao/01-requisitos.md`
- Arquitetura proposta — `documentacao/02-arquitetura.md`
- Documentação da API — `documentacao/03-api.md`
- Plano de testes — `documentacao/04-plano-de-testes.md`
- Diagramas (casos de uso, classes, DER) — pasta `documentacao/diagramas/`

Como executar a demonstração

1. Clonar o repositório e entrar na pasta `backend`;
2. Executar `mvn spring-boot:run` (requer Java 17 e Maven);
3. Acessar a documentação interativa da API em `http://localhost:8080/swagger-ui.html`;
4. Executar os testes automatizados com `mvn test`.

Evidências

- 12 testes automatizados passando (`mvn test`): fluxo de pedido, pagamento, estoque, fidelidade, sazonalidade do cardápio, auditoria e LGPD;
- API documentada automaticamente via OpenAPI/Swagger;
- Trilha de auditoria consultável em GET `/api/auditoria`.

5. Plano de Testes e Evidências

Objetivo

Verificar que o back-end atende aos requisitos funcionais levantados no estudo de caso, com foco nas regras de negócio críticas: cardápio por unidade, ciclo do pedido, integração de pagamento, estoque, fidelidade,

auditoria e LGPD.

Estratégia

Nível	Abordagem	Ferramenta
Integração (serviço + banco)	Testes automatizados @SpringBootTest com H2 em memória e massa de dados do data.sql	JUnit 5 + AssertJ
API (manual/exploratório)	Execução dos endpoints via Swagger UI seguindo os casos abaixo	Swagger UI / curl

Execução automatizada: `cd backend && mvn test.`

Casos de teste

ID	Requisito	Cenário	Resultado esperado
CT01	RF01/RF05	Consultar cardápio da unidade reduzida (SP)	Não lista produtos exclusivos de Recife (bolo de macaxeira, combo)
CT02	RF05	Consultar cardápio fora/dentro do período junino	Canjica junina só aparece entre maio e julho
CT03	RF02/RF12/RF17	Criar pedido de cliente com 120 pontos (2 tapiocas, R\$24)	Pedido AGUARDANDO_PAGAMENTO, desconto 5% (R\$1,20), pagamento solicitado
CT04	RF17/RF06/RF11	Serviço externo confirma o pagamento	Pedido PAG0, estoque baixado, pontos acumulados
CT05	RF18	Resultado do pagamento enviado duas vezes (duplicado)	A repetição é ignorada: estoque e pontos não mudam de novo
CT06	RF17/RF18	Serviço externo recusa o pagamento	Pedido PAGAMENTO_RECUSADO, estoque intacto
CT07	RF07	Cancelar pedido pago	Pedido CANCELADO, estoque devolvido, registro de auditoria com autor/justificativa
CT08	RF07	Aplicar desconto manual	Total recalculado e auditoria de DESCONTO gerada
CT09	RF01	Pedir produto fora do cardápio da unidade	Erro 422 de negócio
CT10	RF06	Pedir quantidade acima do estoque	Erro 422 “Estoque insuficiente”
CT11	RF14/RF11	Registrar consentimento explícito	Consentimento com data/hora e conta de fidelidade criada
CT12	RF16	Consultar dados pessoais de um cliente	Registro de auditoria ACESSO_DADOS_PESSOAIS

ID	Requisito	Cenário	Resultado esperado
CT13	RF15	Anonimizar cliente	Nome/e-mail/data de nascimento apagados definitivamente; auditoria ANONIMIZACAO
CT14	RF04	Avançar status do pedido pago	PAGO → EM_PREPARO → PRONTO → ENTREGUE; transições inválidas retornam 422
CT15	RF08/RF09/RF10/RF20	Consultar relatórios após pedidos confirmados	Vendas por unidade/região, ranking de produtos e indicador meta × realizado coerentes

Os casos CT01 a CT13 são automatizados (classes `FluxoPedidoTest` e `LgpdECardapioTest`, executadas com `mvn test`); CT14 e CT15 são manuais, executados pelo Swagger UI.

Os casos CT01–CT05, CT07, CT08 e CT11–CT15 são **positivos** (comportamento esperado) e CT06, CT09 e CT10 são **negativos** (erros e recusas tratados corretamente).

Exemplos detalhados (entrada, passos e saída esperada)

CT04 — positivo (confirmação de pagamento)

- Entrada: pedido nº 1 em AGUARDANDO_PAGAMENTO; corpo `{"referenciaExterna":"EXT-abc","status":"CONFIRMADO"}`
- Passos: enviar `POST /api/pagamentos/1/resultado`; consultar `GET /api/pedidos/1` e `GET /api/unidades/1/estoque`
- Saída esperada: pedido PAGO, estoque dos itens baixado, pontos do cliente acumulados

CT09 — negativo (produto fora do cardápio)

- Entrada: pedido na unidade 2 (SP) contendo produto exclusivo de Recife
- Passos: enviar `POST /api/pedidos` com o item inválido
- Saída esperada: HTTP 422 com mensagem de negócio; nenhum pedido criado

CT10 — negativo (estoque insuficiente)

- Entrada: pedido com quantidade maior que o estoque da unidade
- Passos: enviar `POST /api/pedidos`
- Saída esperada: HTTP 422 “Estoque insuficiente”; estoque inalterado

Critérios de aceite

- 100% dos testes automatizados passando (`mvn test`);
- casos manuais CT14–CT15 executados via Swagger com evidências;
- nenhuma operação sensível (cancelamento, desconto, ajuste, acesso a dados pessoais, anonimização) sem registro correspondente na trilha de auditoria.

6. Conclusão

O desenvolvimento deste projeto permitiu aplicar, de forma integrada, os conceitos de Engenharia de Software estudados ao longo do curso: levantamento e análise de requisitos a partir de um cenário com ambiguidades, modelagem UML e de dados, definição de arquitetura e implementação de uma API REST funcional com testes automatizados.

Principais lições aprendidas

- Ler o problema antes de codificar: as decisões mais importantes (ciclo do pedido, cardápio por unidade, pagamento externo) vieram da interpretação do estudo de caso, não da tecnologia;

- Delimitar o escopo é tão importante quanto implementar: funcionalidades sem respaldo no estudo de caso (delivery, processamento de cartão, RH) foram deliberadamente excluídas;
- Integrações externas exigem cuidado com falhas: foi preciso garantir que, se o serviço de pagamento enviar o mesmo resultado duas vezes, o sistema não baixe o estoque nem some pontos em dobro.

Desafios

- Traduzir requisitos implícitos (alta disponibilidade, LGPD) em decisões concretas de projeto e código;
- Equilibrar a padronização da franquia com as variações por unidade (cardápio, sazonalidade, receitas regionais).

Pontos de atenção para evoluções futuras

- Migrar o banco H2 (usado para demonstração) para PostgreSQL em produção;
- Adicionar autenticação/autorização por perfil (cliente, atendente, gerente, matriz);
- Integrar um gateway de pagamento real no lugar do simulado;
- Evoluir os relatórios da matriz com filtros por período e exportação.

7. Referências

- Estudo de caso “Rede Raízes do Nordeste — Tecnologia, Tradição e Escala”, disciplina Projeto Multidisciplinar (roteiro oficial da atividade).
- SOMMERVILLE, Ian. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2018.
- Documentação oficial do Spring Boot. Disponível em: <https://spring.io/projects/spring-boot>
- Documentação oficial do Spring Data JPA. Disponível em: <https://spring.io/projects/spring-data-jpa>
- Especificação OpenAPI / Springdoc. Disponível em: <https://springdoc.org>
- PlantUML — ferramenta de diagramação. Disponível em: <https://plantuml.com>
- BRASIL. Lei nº 13.709/2018 (Lei Geral de Proteção de Dados Pessoais — LGPD). Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm

Anexos — Evidências de Execução

Aplicação em execução

Terminal com a API iniciada pelo comando `mvn spring-boot:run` (mensagem “Started RaizesApplication” e Tomcat na porta 8080):

```
PS C:\Users\ramon\Desktop\Raizes-do-nordeste> cd C:\Users\ramon\Desktop\Raizes-do-nordeste\raizes-do-nordeste-backend\backend
>> mvn spring-boot:run
2026-08-03T21:31:05.582-03:00 INFO 34528 --- [raizes-backend] [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA pl
atform integration)
2026-08-03T21:31:05.587-03:00 INFO 34528 --- [raizes-backend] [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-08-03T21:31:06.007-03:00 INFO 34528 --- [raizes-backend] [main] o.s.d.j.r.query.QueryEnhancerFactory : Hibernate is in classpath, if applicable, H2 parser will be used.
2026-08-03T21:31:07.472-03:00 INFO 34528 --- [raizes-backend] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2026-08-03T21:31:07.482-03:00 INFO 34528 --- [raizes-backend] [main] b.c.raizesdonordeste.RaizesApplication : Started RaizesApplication in 6.053 seconds (process running for 6.411)
2026-08-03T21:31:29.014-03:00 INFO 34528 --- [raizes-backend] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-08-03T21:31:29.014-03:00 INFO 34528 --- [raizes-backend] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-08-03T21:31:29.016-03:00 INFO 34528 --- [raizes-backend] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms
2026-08-03T21:31:29.922-03:00 INFO 34528 --- [raizes-backend] [nio-8080-exec-9] o.springdoc.api.AbstractOpenApiResource : Init duration for springdoc-openapi is: 647 ms
```

Figura 4: Aplicação em execução

Swagger UI

Com a API rodando, a documentação interativa fica disponível em <http://localhost:8080/swagger-ui.html>:



Figura 5: Swagger UI

Exemplo de requisição

Consulta do cardápio da unidade 1 no navegador (GET /api/unidades/1/cardapio), retornando os produtos disponíveis na unidade:

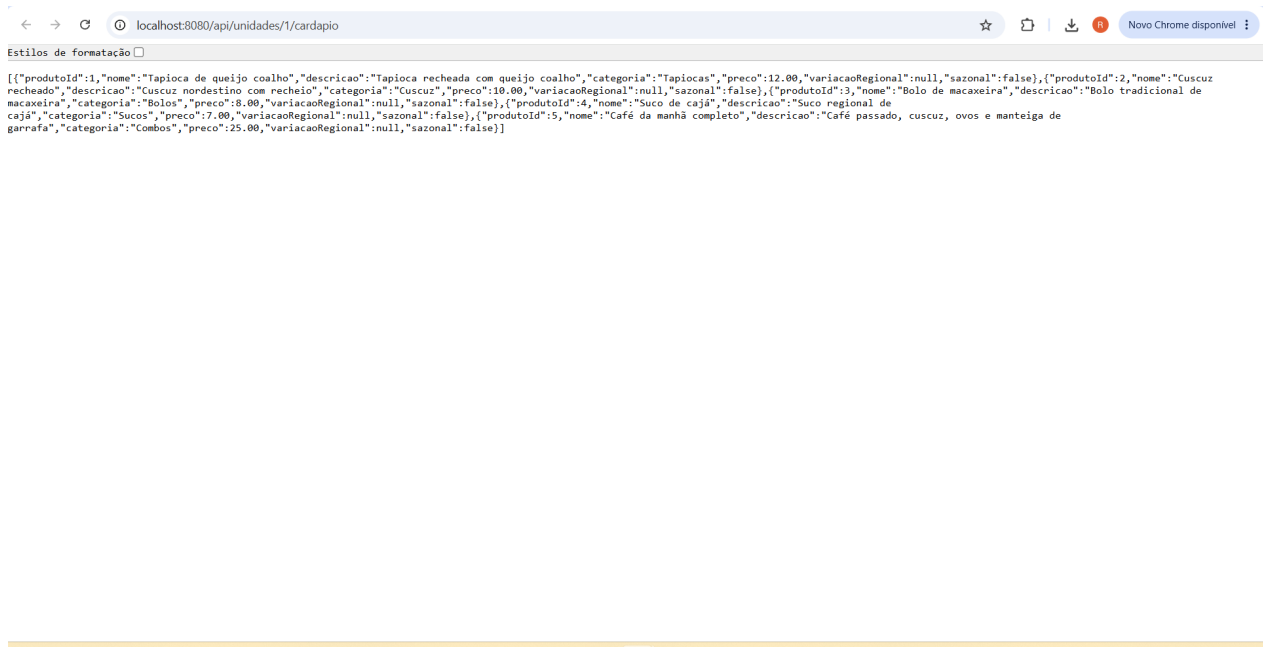


Figura 6: Cardápio da unidade 1